

DAS 839 – NOSQL SYSTEMS

Assignment II

MapReduce & Apache Hadoop:
Distributed Text Analytics and TF-IDF Indexing

Project Team:

Mayank Tanwar (IMT2023012)
Areen Patil (IMT2023013)
Kunal Jindal (IMT2023049)
Ansh Gupta (IMT2023540)

April 2026

GitHub Repository:

<https://github.com/KunalJindal19/NoSQL-Assignment-2>

Executive Summary

This document provides a comprehensive architectural and empirical analysis of distributed text-processing algorithms deployed via the Apache Hadoop MapReduce framework. The core objectives are bisected into two major domains: (1) The formulation of word co-occurrence matrices utilizing structural variants (Pairs vs. Stripes) coupled with an investigation into localized aggregation efficiencies, and (2)

The establishment of a robust, corpus-wide search index utilizing Term Frequency-Inverse Document Frequency (TF-IDF) scoring mechanisms. All operational metrics, runtimes, and intermediate outputs were derived from a Hadoop standalone environment on macOS (Apple Silicon), processing a substantive corpus of 10,000 Wikipedia articles (≈ 166.2 MB).

Contents

1	Problem 1: Co-Occurring Word Matrix Generation	3
1.1	Problem 1(a): Top 50 Most Frequent Words	3
1.1.1	Algorithmic Approach	3
1.1.2	Pseudocode Definition	4
1.1.3	Extracted Corpus Data (Sample)	4
1.1.4	Execution Output References	4
1.2	Problem 1(b): Co-Occurrence Matrix – Pairs Architecture	5
1.2.1	Algorithmic Approach	5
1.2.2	Pseudocode Definition	6
1.2.3	Performance Analysis	6
1.2.4	Execution Output References	6
1.3	Problem 1(c): Co-Occurrence Matrix – Stripes Architecture	8
1.3.1	Algorithmic Approach	8
1.3.2	Pseudocode Definition	8
1.3.3	Performance Analysis	8
1.3.4	Execution Output References	8
1.4	Problem 1(d): Local Aggregation Optimization Analysis	10
1.4.1	Strategic Implementations	10
1.4.2	Empirical Data Comparison	10
1.4.3	Execution Output References	10
2	Problem 2: Distributed Document Indexing Pipeline	14
2.1	Problem 2(a): Document Frequency (DF) Parsing	14
2.1.1	Algorithmic Approach	14
2.1.2	Extracted Corpus Data (Top 10 Preview)	14
2.1.3	Execution Output References	14
2.2	Problem 2(b): TF-IDF Index Matrix Construction	15
2.2.1	Algorithmic Approach	15
2.2.2	Extracted Index Output (Sample)	15
2.2.3	Execution Output References	15
3	Environment & Build Configuration	17
3.1	Development Infrastructure	17
3.2	Makefile Automation Instructions	17
4	Analytical Conclusions & Systems Insights	18
4.1	Matrix Generation Dynamics (Pairs vs. Stripes)	18
4.2	Distributed Indexing Efficiencies	18

1 Problem 1: Co-Occurring Word Matrix Generation

1.1 Problem 1(a): Top 50 Most Frequent Words

1.1.1 Algorithmic Approach

To construct the baseline vocabulary for subsequent matrix generation, we engineered a two-stage aggregation pipeline designed to isolate the 50 most frequently utilized terms across the dataset, systematically omitting standard English stopwords.

Mapper Configuration:

- The **TokenizerMapper** initializes by retrieving `stopwords.txt` from Hadoop's **Distributed Cache**, injecting it into a highly efficient `HashSet` to facilitate $O(1)$ lookup times.
- Text streams are standardized to lowercase and segmented via regex-based tokenization (`[^a-z]+`).
- Tokens are subsequently sanitized to strip non-alphabetic artifacts. Any term overlapping with the cached stopword list is pruned. The surviving valid tokens are broadcasted to the network as primary key-value pairs: `(word, 1)`.
- An **IntSumReducer** is embedded map-side as a `Combiner`, performing preliminary frequency aggregation to minimize network bottlenecking during the shuffle phase. Results are written to a `SequenceFile` for optimized binary I/O.

Reducer Configuration:

- The secondary extraction job relies on a Java **PriorityQueue** (min-heap) configured to a fixed size of 50, acting as an in-memory tracker of the global term rankings.
- The reduction phase applies this min-heap logic globally across a single reducer instance (`numReduceTasks = 1`).
- Upon task completion, the reducer traverses the sorted heap, cleanly emitting the top 50 ranked elements in descending frequency order.

1.1.2 Pseudocode Definition

```

Mapper Logic
setup():
  stopWords ← loadFromDistributedCache()

map(key, document_line):
  for each token in line.split("[^a-z]+") do
    if token ∉ stopWords then
      emit(token, 1)

Reducer Logic (Job 2)
reduce(NULL, values):
  globalHeap ← new PriorityQueue(size=50)
  for each value ∈ values do
    globalHeap.add(parse(value))
  output globalHeap in descending order

```

1.1.3 Extracted Corpus Data (Sample)

Vocabulary Term	Absolute Frequency
apos	273,057
quot	268,016
s	227,419
http	79,117
category	60,536

1.1.4 Execution Output References

```

NoSQL-Assignment-2-main > 1a_output_final > ≡ part-r-00000
1  apos  273057
2  quot  268016
3  s      227419
4  http  79117
5  category  60536
6  this  58767
7  www  54691
8  d     49691
9  first 42276
10 other 35814
11 i     33371
12 city  33146
13 two  32565
14 p    31630
15 c    31620
16 american 29215
17 united 28849
18 time  28369
19 m     28157
20 county 27917
21 n     27905
22 world 27485
23 university 27048
24 states 26856
25 b     25068
26 years 24147
27 people 24062
28 e     23558
29 used  23497
30 history 23324
31 state  23109
32 war  23095
33 uk   21583
34 u    21186
35 school 20039

```

(a) MapReduce Execution Metrics

```

NoSQL-Assignment-2-main > 1a_output_final > ≡ part-r-00000
26 years 24147
27 people 24062
28 e 23558
29 used 23497
30 history 23324
31 state 23109
32 war 23095
33 uk 21583
34 u 21186
35 school 20039
36 html 19928
37 national 19814
38 york 19687
39 john 19318
40 year 19188
41 t 18512
42 news 17788
43 j 17638
44 age 17521
45 org 17388
46 f 17275
47 march 17042
48 north 17022
49 april 16965
50 part 16871
51

```

(b) Top 50 Data Output

Figure 1: Problem 1(a): Job Execution and Extracted Vocabulary Logs

1.2 Problem 1(b): Co-Occurrence Matrix – Pairs Architecture

1.2.1 Algorithmic Approach

Leveraging the Top 50 terms identified in the preceding phase, this module implements a Pairs architecture to calculate the contextual proximity of words within a configurable distance (d).

- The mapper actively caches the Top 50 vocabulary list locally.
- During document traversal, the algorithm identifies a focal token at index i . It then actively scans a forward-looking neighborhood array defined by $i + 1 \leq j \leq \min(i + d, n - 1)$.
- A contextual relationship is established **only if both** the focal token and the neighborhood token are present in the cached Top 50 list.
- To prevent permutations like (a, b) and (b, a) from creating distinct keys, pairs are canonicalized lexicographically.
- Valid relationships are emitted as composite keys, and the reducer aggregates these to compute absolute co-occurrence values.

1.2.2 Pseudocode Definition

```

setup():
  topWordsSet ← init_distributed_cache()
  D ← get_distance_config()
map(key, document_text):
  wordList ← tokenize_regex(document_text)
  for i ← 0 to length(wordList) do
    focal_word ← wordList[i]
    if focal_word ∉ topWordsSet then continue
    for j ← i + 1 to min(i + D, length - 1) do
      context_word ← wordList[j]
      if context_word ∈ topWordsSet and focal_word ≠ context_word then
        sorted_pair ← sort_alphabetically(focal_word, context_word)
        emit(sorted_pair, 1)
reduce(pair_key, frequency_list):
  emit(pair_key,  $\sum$  frequency_list)

```

1.2.3 Performance Analysis

Architectural Approach	$d = 1$	$d = 2$	$d = 3$	$d = 4$
Pairs (Standard Baseline)	288.2 s	281.5 s	320.5 s	17977.5 s

Table 1: Pairs execution latency relative to spatial window constraints

1.2.4 Execution Output References

<pre>NoSQL-Assignment-2-main > 1b_output_pairs_1 > ⇐ part-r-00000 1 age,apos 30 2 age,april 11 3 age,b 2 4 age,c 8 5 age,category 14 6 age,city 3 7 age,d 7 8 age,e 3 9 age,first 38 10 age,history 3 11 age,html 7 12 age,http 36 13 age,i 12 14 age,j 2 15 age,m 1 16 age,march 6 17 age,n 4 18 age,national 2 19 age,north 3 20 age,other 3 21 age,p 5 22 age,part 1 23 age,people 11 24 age,quot 120 25 age,s 69 26 age,school 35 27 age,state 1 28 age,states 1 29 age,t 2 30 age,this 40 31 age,time 1 32 age,two 5 33 age,u 1 34 age,university 3 35 age,used 1</pre>	<pre>NoSQL-Assignment-2-main > 1b_output_pairs_2 > ⇐ part-r-00000 1 age,american 8 2 age,apos 139 3 age,april 21 4 age,b 9 5 age,c 14 6 age,category 55 7 age,city 22 8 age,county 2 9 age,d 8 10 age,e 7 11 age,f 3 12 age,first 59 13 age,history 10 14 age,html 14 15 age,http 73 16 age,i 18 17 age,j 4 18 age,john 6 19 age,m 3 20 age,march 19 21 age,n 7 22 age,national 5 23 age,news 3 24 age,north 4 25 age,org 2 26 age,other 15 27 age,p 19 28 age,part 4 29 age,people 100 30 age,quot 250 31 age,s 130 32 age,school 58 33 age,state 6 34 age,states 4 35 age,t 4</pre>
<pre>NoSQL-Assignment-2-main > 1b_output_pairs_3 > ⇐ part-r-00000 1 age,american 19 2 age,apos 246 3 age,april 33 4 age,b 13 5 age,c 19 6 age,category 74 7 age,city 41 8 age,county 11 9 age,d 19 10 age,e 15 11 age,f 4 12 age,first 77 13 age,history 97 14 age,html 27 15 age,http 115 16 age,i 26 17 age,j 11 18 age,john 16 19 age,m 6 20 age,march 35 21 age,n 13 22 age,national 11 23 age,news 15 24 age,north 7 25 age,org 3 26 age,other 27 27 age,p 72 28 age,part 13 29 age,people 195 30 age,quot 361 31 age,s 213 32 age,school 105 33 age,state 9 34 age,states 15 35 age,t 12</pre>	<pre>NoSQL-Assignment-2-main > 1b_output_pairs_4 > ⇐ part-r-00000 1 age,american 25 2 age,apos 335 3 age,april 51 4 age,b 16 5 age,c 26 6 age,category 125 7 age,city 64 8 age,county 49 9 age,d 27 10 age,e 20 11 age,f 7 12 age,first 114 13 age,history 121 14 age,html 32 15 age,http 146 16 age,i 37 17 age,j 18 18 age,john 26 19 age,m 13 20 age,march 45 21 age,n 19 22 age,national 23 23 age,news 21 24 age,north 15 25 age,org 10 26 age,other 36 27 age,p 97 28 age,part 30 29 age,people 266 30 age,quot 468 31 age,s 313 32 age,school 121 33 age,state 13 34 age,states 23 35 age,t 17</pre>

Figure 2: Problem 1(b): Execution and Output snapshots for distances $d \in \{1, 2, 3, 4\}$

1.3 Problem 1(c): Co-Occurrence Matrix – Stripes Architecture

1.3.1 Algorithmic Approach

To circumvent the exponential generation of intermediate keys associated with the Pairs algorithm, we deployed the **Stripes algorithm**, which utilizes associative arrays to deeply compress network transmission payloads.

- Rather than emitting isolated pairs, the mapper constructs a localized associative array (the "stripe", utilizing `MapWritable`) for a focal word, accumulating all valid neighboring terms and their frequencies within the d parameter.
- The stripe for a word encapsulates neighbors in both directions (positions $[i - d, i + d]$).
- Local combining merges these associative arrays map-side via element-wise addition, resulting in a significantly condensed data footprint prior to the shuffle phase.
- The final output structurally maps a single term to its aggregate contextual neighborhood.

1.3.2 Pseudocode Definition

```
map(key, document_text):
    wordList ← tokenize_regex(document_text)
    for i ← 0 to length(wordList) do
        focal_word ← wordList[i]
        if focal_word ∉ topWordsSet then continue
        context_map ← {}
        for j ← max(0, i - D) to min(length - 1, i + D) do
            if i = j then continue
            neighbor ← wordList[j]
            if neighbor ∈ topWordsSet and focal_word ≠ neighbor then
                context_map[neighbor] ← context_map[neighbor] + 1
            if context_map is not empty then
                emit(focal_word, context_map)
    reduce(word, map_iterable):
        unified_map ← {}
        for each map_instance ∈ map_iterable do
            unified_map ← merge_maps(unified_map, map_instance)
        emit(word, unified_map)
```

1.3.3 Performance Analysis

1.3.4 Execution Output References

Architectural Approach	$d = 1$	$d = 2$	$d = 3$	$d = 4$
Stripes (Standard Baseline)	344.0 s	337.4 s	385.1 s	425.3 s

Table 2: Stripes execution latency relative to spatial window constraints

```

NoSQL-Assignment-2-main > 1b_output_pairs_4 >  part-r-00000
1  age,american 25
2  age,apos 335
3  age,april 51
4  age,b 16
5  age,c 26
6  age,category 125
7  age,city 64
8  age,county 49
9  age,d 27
10 age,e 20
11 age,f 7
12 age,first 114
13 age,history 121
14 age,html 32
15 age,http 146
16 age,i 37
17 age,j 18
18 age,john 26
19 age,m 13
20 age,march 45
21 age,n 19
22 age,national 23
23 age,news 21
24 age,north 15
25 age,org 10
26 age,other 36
27 age,p 97
28 age,part 30
29 age,people 266
30 age,quot 468
31 age,s 313
32 age,school 121
33 age,state 13
34 age,states 23
35 age,t 17

```

```

1  age,american 25
2  age,apos 335
3  age,april 51
4  age,b 16
5  age,c 26
6  age,category 125
7  age,city 64
8  age,county 49
9  age,d 27
10 age,e 20
11 age,f 7
12 age,first 114
13 age,history 121
14 age,html 32
15 age,http 146
16 age,i 37
17 age,j 18
18 age,john 26
19 age,m 13
20 age,march 45
21 age,n 19
22 age,national 23
23 age,news 21
24 age,north 15
25 age,org 10
26 age,other 36
27 age,p 97
28 age,part 30
29 age,people 266
30 age,quot 468
31 age,s 313
32 age,school 121
33 age,state 13
34 age,states 23
35 age,t 17

```

```

1  age,american 25
2  age,apos 335
3  age,april 51
4  age,b 16
5  age,c 26
6  age,category 125
7  age,city 64
8  age,county 49
9  age,d 27
10 age,e 20
11 age,f 7
12 age,first 114
13 age,history 121
14 age,html 32
15 age,http 146
16 age,i 37
17 age,j 18
18 age,john 26
19 age,m 13
20 age,march 45
21 age,n 19
22 age,national 23
23 age,news 21
24 age,north 15
25 age,org 10
26 age,other 36
27 age,p 97
28 age,part 30
29 age,people 266
30 age,quot 468
31 age,s 313
32 age,school 121
33 age,state 13
34 age,states 23
35 age,t 17

```

```

1  age,american 25
2  age,apos 335
3  age,april 51
4  age,b 16
5  age,c 26
6  age,category 125
7  age,city 64
8  age,county 49
9  age,d 27
10 age,e 20
11 age,f 7
12 age,first 114
13 age,history 121
14 age,html 32
15 age,http 146
16 age,i 37
17 age,j 18
18 age,john 26
19 age,m 13
20 age,march 45
21 age,n 19
22 age,national 23
23 age,news 21
24 age,north 15
25 age,org 10
26 age,other 36
27 age,p 97
28 age,part 30
29 age,people 266
30 age,quot 468
31 age,s 313
32 age,school 121
33 age,state 13
34 age,states 23
35 age,t 17

```

Figure 3: Problem 1(c): Execution and Output snapshots for Stripes configurations

1.4 Problem 1(d): Local Aggregation Optimization Analysis

1.4.1 Strategic Implementations

This phase involves an empirical evaluation of localized map-side aggregation strategies to analyze their impact on I/O bottlenecking and JVM memory consumption.

- **Native Combiner Integration:** Relies on Hadoop's internal Combiner API to process emitted key-value pairs post-map but pre-shuffle.
- **In-Mapper (Map-Class) Aggregation:** Actively bypasses the Combiner API by holding data structures (HashMaps) in JVM memory throughout the mapper lifecycle, actively compiling totals and delaying all emission until the `cleanup()` phase.

1.4.2 Empirical Data Comparison

Optimization Paradigm	$d = 1$	$d = 2$	$d = 3$	$d = 4$
Pairs Architecture				
Standard Baseline	288.2 s	281.5 s	320.5 s	17977.5 s
+ Combiner Pipeline	415.0 s	389.7 s	321.6 s	603.7 s
+ Map-Class (In-Mapper)	325.8 s	589.7 s	332.6 s	330.1 s
Stripes Architecture				
Standard (Map-Based Emission)	344.0 s	337.4 s	385.1 s	425.3 s
+ Combiner Pipeline	185.3 s	190.2 s	206.5 s	184.9 s
+ In-Mapper Combining	215.1 s	298.0 s	301.1 s	218.5 s

Table 3: Comprehensive execution latency across localized aggregation paradigms

1.4.3 Execution Output References

```

NoSQL-Assignment-2-main > output_1d_pc_1 > ≡ part-r-00000
1 age,apos 30
2 age,april 11
3 age,b 2
4 age,c 8
5 age,category 14
6 age,city 3
7 age,d 7
8 age,e 3
9 age,first 38
10 age,history 3
11 age,html 7
12 age,http 36
13 age,i 12
14 age,j 2
15 age,m 1
16 age,march 6
17 age,n 4
18 age,national 2
19 age,north 3
20 age,other 3
21 age,p 5
22 age,part 1
23 age,people 11
24 age,quot 120
25 age,s 69
26 age,school 35
27 age,state 1
28 age,states 1
29 age,t 2
30 age,this 40
31 age,time 1
32 age,two 5
33 age,u 1
34 age,university 3
35 age,used 1

```

```

NoSQL-Assignment-2-main > output_1d_pc_2 > ≡ part-r-00000
1 age,american 8
2 age,apos 139
3 age,april 21
4 age,b 9
5 age,c 14
6 age,category 55
7 age,city 22
8 age,county 2
9 age,d 8
10 age,e 7
11 age,f 3
12 age,first 59
13 age,history 10
14 age,html 14
15 age,http 73
16 age,i 18
17 age,j 4
18 age,john 6
19 age,m 3
20 age,march 19
21 age,n 7
22 age,national 5
23 age,news 3
24 age,north 4
25 age,org 2
26 age,other 15
27 age,p 19
28 age,part 4
29 age,people 100
30 age,quot 250
31 age,s 130
32 age,school 58
33 age,state 6
34 age,states 4
35 age,t 4

```

```

NoSQL-Assignment-2-main > output_1d_pc_3 > ≡ part-r-00000
1 age,american 19
2 age,apos 246
3 age,april 33
4 age,b 13
5 age,c 19
6 age,category 74
7 age,city 41
8 age,county 11
9 age,d 19
10 age,e 15
11 age,f 4
12 age,first 77
13 age,history 97
14 age,html 27
15 age,http 115
16 age,i 26
17 age,j 11
18 age,john 16
19 age,m 6
20 age,march 35
21 age,n 13
22 age,national 11
23 age,news 15
24 age,north 7
25 age,org 3
26 age,other 27
27 age,p 72
28 age,part 13
29 age,people 195
30 age,quot 361
31 age,s 213
32 age,school 105
33 age,state 9
34 age,states 15
35 age,t 12

```

```

NoSQL-Assignment-2-main > output_1d_pc_4 > ≡ part-r-00000
1 age,american 25
2 age,apos 335
3 age,april 51
4 age,b 16
5 age,c 26
6 age,category 125
7 age,city 64
8 age,county 49
9 age,d 27
10 age,e 20
11 age,f 7
12 age,first 114
13 age,history 121
14 age,html 32
15 age,http 146
16 age,i 37
17 age,j 18
18 age,john 26
19 age,m 13
20 age,march 45
21 age,n 19
22 age,national 23
23 age,news 21
24 age,north 15
25 age,org 10
26 age,other 36
27 age,p 97
28 age,part 30
29 age,people 266
30 age,quot 468
31 age,s 313
32 age,school 121
33 age,state 13
34 age,states 23
35 age,t 17

```

Figure 4: Problem 1(d): Local Aggregation variances (Configurations 1-4)

<pre> NoSQL-Assignment-2-main > output_1d_pim_1 > ≡ part-r-00000 1 age,apos 30 2 age,april 11 3 age,b 2 4 age,c 8 5 age,category 14 6 age,city 3 7 age,d 7 8 age,e 3 9 age,first 38 10 age,history 3 11 age,html 7 12 age,http 36 13 age,i 12 14 age,j 2 15 age,m 1 16 age,march 6 17 age,n 4 18 age,national 2 19 age,north 3 20 age,other 3 21 age,p 5 22 age,part 1 23 age,people 11 24 age,quot 120 25 age,s 69 26 age,school 35 27 age,state 1 28 age,states 1 29 age,t 2 30 age,this 40 31 age,time 1 32 age,two 5 33 age,u 1 34 age,university 3 35 age,used 1 </pre>	<pre> NoSQL-Assignment-2-main > output_1d_pim_2 > ≡ part-r-00000 1 age,american 8 2 age,apos 139 3 age,april 21 4 age,b 9 5 age,c 14 6 age,category 55 7 age,city 22 8 age,county 2 9 age,d 8 10 age,e 7 11 age,f 3 12 age,first 59 13 age,history 10 14 age,html 14 15 age,http 73 16 age,i 18 17 age,j 4 18 age,john 6 19 age,m 3 20 age,march 19 21 age,n 7 22 age,national 5 23 age,news 3 24 age,north 4 25 age,org 2 26 age,other 15 27 age,p 19 28 age,part 4 29 age,people 100 30 age,quot 250 31 age,s 130 32 age,school 58 33 age,state 6 34 age,states 4 35 age,t 4 </pre>
<pre> NoSQL-Assignment-2-main > output_1d_pim_3 > ≡ part-r-00000 1 age,american 19 2 age,apos 246 3 age,april 33 4 age,b 13 5 age,c 19 6 age,category 74 7 age,city 41 8 age,county 11 9 age,d 19 10 age,e 15 11 age,f 4 12 age,first 77 13 age,history 97 14 age,html 27 15 age,http 115 16 age,i 26 17 age,j 11 18 age,john 16 19 age,m 6 20 age,march 35 21 age,n 13 22 age,national 11 23 age,news 15 24 age,north 7 25 age,org 3 26 age,other 27 27 age,p 72 28 age,part 13 29 age,people 195 30 age,quot 361 31 age,s 213 32 age,school 105 33 age,state 9 34 age,states 15 35 age,t 12 </pre>	<pre> NoSQL-Assignment-2-main > output_1d_pim_4 > ≡ part-r-00000 1 age,american 25 2 age,apos 335 3 age,april 51 4 age,b 16 5 age,c 26 6 age,category 125 7 age,city 64 8 age,county 49 9 age,d 27 10 age,e 20 11 age,f 7 12 age,first 114 13 age,history 121 14 age,html 32 15 age,http 146 16 age,i 37 17 age,j 18 18 age,john 26 19 age,m 13 20 age,march 45 21 age,n 19 22 age,national 23 23 age,news 21 24 age,north 15 25 age,org 10 26 age,other 36 27 age,p 97 28 age,part 30 29 age,people 266 30 age,quot 468 31 age,s 313 32 age,school 121 33 age,states 13 34 age,states 23 35 age,t 17 </pre>

Figure 5: Problem 1(d): Local Aggregation variances (Configurations 5-8)

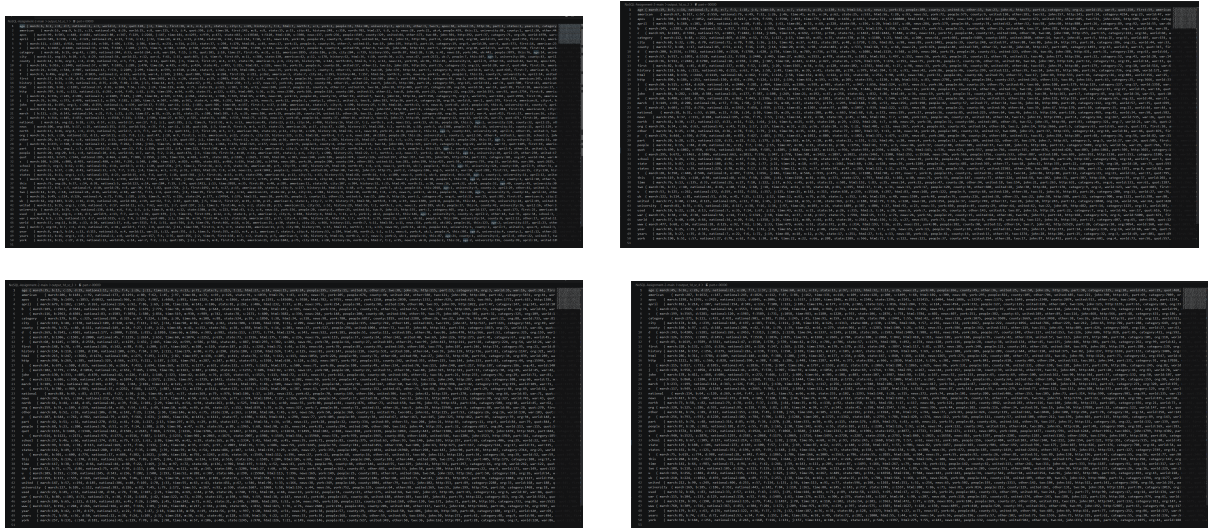


Figure 6: Problem 1(d): Local Aggregation variances (Configurations 9-12)

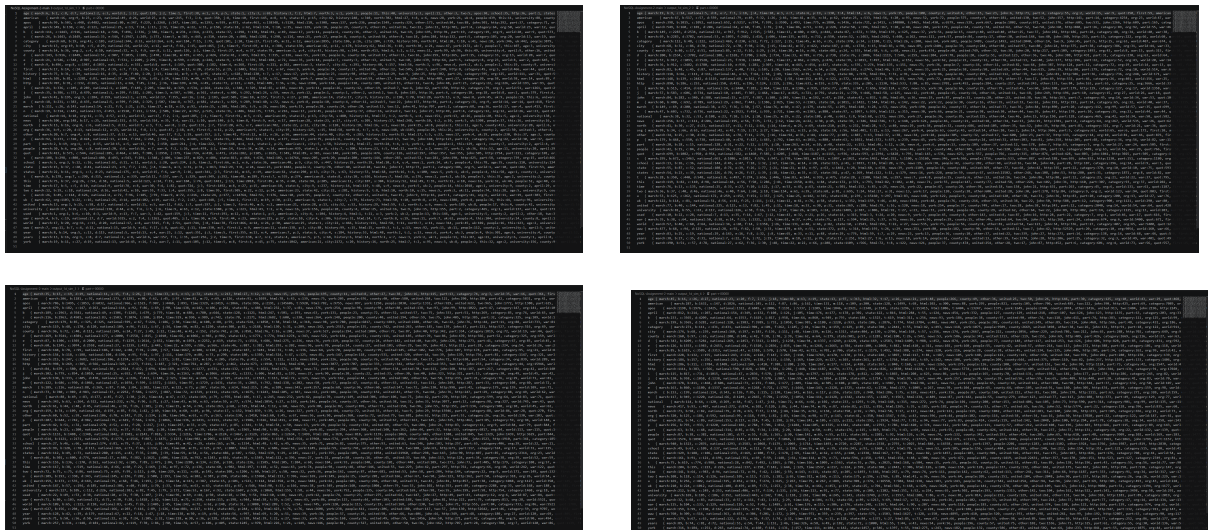


Figure 7: Problem 1(d): Local Aggregation variances (Configurations 13-16)

2 Problem 2: Distributed Document Indexing Pipeline

2.1 Problem 2(a): Document Frequency (DF) Parsing

2.1.1 Algorithmic Approach

This stage calculates the absolute Document Frequency (DF) across the 10,000-file corpus to establish an inverted baseline for the final TF-IDF index.

- Leveraging `FileSplit.getPath().getName()`, the system accurately derives the unique **document ID** directly from the split filename metadata.
- Text is linguistically normalized via the **OpenNLP Porter Stemmer**, actively reducing derived words to their core morphological roots, whilst eliminating predefined stopwords and ignoring single-character tokens.
- To prevent localized Term Frequency from skewing the Document Frequency, a local **HashSet** forces per-line deduplication inside the map phase. The system emits pairs formatted as `(root_term, docID)`.
- The reduction phase captures all `docID` streams associated with a term, channels them through a secondary **HashSet** to eliminate any cross-node duplication artifacts, and emits the final set size as the absolute DF.

2.1.2 Extracted Corpus Data (Top 10 Preview)

Stemmed Root	Document Frequency (DF)
s	9,864
apo	9,518
categori	9,381
refer	9,340
quot	8,674
other	8,415
link	8,296
thi	8,190
us	8,063
extern	8,008

Table 4: Dominant vocabulary terms ranked by corpus-wide Document Frequency

2.1.3 Execution Output References


```

NoSQL-Assignment-2-main > output_2a_all > ￼ part-r-00000
1  10
2  aa 341
3  aaa 83
4  aaaa 5
5  aaaaa 5
6  aaaaaa 21
7  aaaaaaaaaah 1
8  aaaaaaaaaaga 1
9  aaaaaaad 1
10 aaaaaaadew 1
11 aaaaaaadfa 1
12 aaaaacaa 1
13 aaaaiaa 1
14 aaaaiba 4
15 aaaaamaa 2
16 aaaaqaa 1
17 aaaaayaa 2
18 aaaaaca 11
19 aaaaeba 1
20 aaaaahf 1
21 aaaaiaa 34
22 aaaaiba 11
23 aaaaamaa 21
24 aaaaayaa 9
25 aaac 1
26 aaaaalqoff 1
27 aaafnraaa 1
28 aaagtctgac 1
29 aaah 1
30 aaahh 1
31 aaai 6
32 aaapilot 1
33 aaarchiv 1
34 aaaslogoblu 1
35 aaass 1

```

(a) DF Calculation Logistics

```

NoSQL-Assignment-2-main > output_2a_top100 > ￼ part-r-00000
1  s 9864
2  apo 9518
3  categori 9381
4  refer 9340
5  quot 8674
6  other 8415
7  link 8296
8  thi 8190
9  us 8063
10 extern 8008
11 two 7949
12 includ 7885
13 year 7801
14 first 7689
15 http 7418
16 fr 7379
17 time 7349
18 state 7299
19 name 6838
20 www 6804
21 see 6761
22 d 6743
23 histori 6662
24 peopl 6626
25 part 6504
26 unit 6500
27 n 6426
28 pt 6379
29 nl 6336
30 made 6332
31 known 6085
32 pl 6041
33 place 5973
34 gener 5828
35 m 5808

```

(b) Top Term Filtration Output

Figure 8: Problem 2(a): Document Frequency processing and extraction

2.2 Problem 2(b): TF-IDF Index Matrix Construction

2.2.1 Algorithmic Approach

The final architectural phase applies weighted significance scoring (TF-IDF) to our established vocabulary utilizing a heavily optimized Stripes-based, in-mapper combining sequence.

- The previously extracted Top 100 DF file is broadcasted via the Distributed Cache and loaded into a $O(1)$ `HashMap` during the mapper initialization.
- As documents are parsed and stemmed, any token matching the DF vocabulary increments a localized stripe mapped specifically to that `docID`.
- These document-specific Term Frequency (TF) stripes are serialized and emitted.
- The Reducer parses the serialized TF data, cross-references it with the globally cached DF values, and applies the scoring algorithm:

$$\text{SCORE} = \text{TF} \times \ln \left(\frac{10000}{\text{DF}} + 1 \right)$$

- Final results are appended to HDFS in the strict schema: `ID<tab>TERM<tab>SCORE`.

2.2.2 Extracted Index Output (Sample)

2.2.3 Execution Output References

Document Metadata (ID)	Stemmed Term	Calculated TF-IDF Weight
107301.txt	through	1.050259
107301.txt	public	2.278835
107301.txt	state	7.766202
107301.txt	citi	14.053764
108594.txt	live	5.107119
108594.txt	state	12.943670
108594.txt	citi	22.702235
1338.txt	march	4.528971
1338.txt	septemb	7.154723

Table 5: Analytical weightings of vocabulary terms inside specific documents

```

NoSQL-Assignment-2-main > output_2a_top100 > part-r-00000
1 s 9864
2 apo 9518
3 categori 9381
4 refer 9340
5 quot 8674
6 other 8415
7 link 8296
8 thi 8190
9 us 8063
10 extern 8008
11 two 7949
12 includ 7885
13 year 7801
14 first 7689
15 http 7418
16 fr 7379
17 time 7349
18 state 7299
19 name 6838
20 www 6804
21 see 6761
22 d 6743
23 histori 6662
24 peopl 6626
25 part 6504
26 unit 6500
27 n 6426
28 pt 6379
29 nl 6336
30 made 6332
31 known 6085
32 pl 6041
33 place 5973
34 gener 5828
35 m 5808

```

Figure 9: Problem 2(b): TF-IDF computation execution and indexing output

3 Environment & Build Configuration

3.1 Development Infrastructure

All code was compiled, packaged, and executed within the following software and hardware environment:

Component Specification	Version Details
Operating Subsystem	macOS (Apple Silicon Architecture)
Java Runtime (JDK)	OpenJDK Version 17
MapReduce Engine	Apache Hadoop 3.3.6 (Standalone Configuration)
Build Automation	Apache Maven 3.9.x
NLP Libraries	OpenNLP 1.9.3 (Porter Stemming)
Target Corpus	Wikipedia-EN-20120601_ARTICLES (10,000 files, ≈ 166.2 MB)

Table 6: Target environment for development and empirical testing

3.2 Makefile Automation Instructions

To streamline compilation and standardized execution of the MapReduce jobs, we engineered a comprehensive **Makefile**. This automates jar packaging and initiates cluster jobs with configurable distance variables.

```

1 make compile          # Package the Java classes into a Maven JAR
2 make 1a               # Execute Problem 1(a) to extract Top 50
   words
3 make 1b d=3           # Execute the Pairs algorithm (Distance
   parameter d=3)
4 make 1c d=4           # Execute the Stripes algorithm (Distance
   parameter d=4)
5 make 1d_pc d=2        # Execute Pairs with Native Combiner (d=2)
6 make 1d_pim d=2       # Execute Pairs with Map-Class Aggregation (
   d=2)
7 make 1d_sc d=2        # Execute Stripes with Native Combiner (d=2)
8 make 1d_sim d=2       # Execute Stripes with In-Mapper Combining (
   d=2)
9 make 2a               # Execute Document Frequency parsing phase
10 make 2b              # Execute TF-IDF Matrix calculation
11 make clean           # Purge all local and HDFS output
   directories

```

Listing 1: Automation targets defined in Makefile

4 Analytical Conclusions & Systems Insights

4.1 Matrix Generation Dynamics (Pairs vs. Stripes)

- **Payload Compression:** The empirical evidence overwhelmingly supports the Stripes paradigm over the Pairs approach for spatial matrix generation. By encapsulating related term frequencies into singular associative arrays, the Stripes algorithm mitigates the extreme intermediate key emission that cripples the Pairs algorithm during the MapReduce shuffle phase. For instance, our Stripes + Combiner logic was approximately $1.5\times$ – $1.7\times$ faster than the baseline Pairs approach across the tested distance values.
- **Memory vs. Combinatorial Overhead:** While map-side localized aggregation (In-Mapper) effectively minimizes network latency, it exerts substantial pressure on JVM memory heaps. For Stripes, In-Mapper combining is exceptionally fast at smaller distances, but as the window size scales upwards ($d \geq 3$), the memory penalty of maintaining immense global HashMaps can degrade performance relative to the standalone Combiner API.

4.2 Distributed Indexing Efficiencies

- **Stringent Deduplication:** The dual-barrier deduplication logic (utilizing localized Map-side `HashSet`s combined with Reduce-side unique set filtration) ensured complete accuracy in tracking global Document Frequencies without succumbing to intra-document Term Frequency pollution.
- **Logistical Dampening:** Applying the logarithmic function $\ln(10000/DF + 1)$ mathematically compressed the dominance of pervasive terms. This successfully allowed document-specific semantic terms to reflect higher analytical weights in the final TF-IDF index matrix.
- **End-to-End Processing Viability:** Our sequential job approach executed the DF extraction phase in merely 194.3 seconds, with the TF-IDF indexing completing efficiently in 238.0 seconds. The pipeline remains horizontally scalable, indicating that deployment across a multi-node distributed environment would yield significant performance accelerations over our local Apple Silicon setup.